面向对象与开源项目分析

AI辅助代码理解 | 开源项目结构分析

从“会写一个类”走向“读懂一个真实软件系统”

徐迎晓

复旦大学计算与智能创新学院

xuyingxiao@126.com

从“会写类”到“会读系统”

入门阶段

写一个类

- 关注语法是否正确
- 对象数量少、关系可见
- 入口清楚、代码短
- 教师通常提前设计好边界

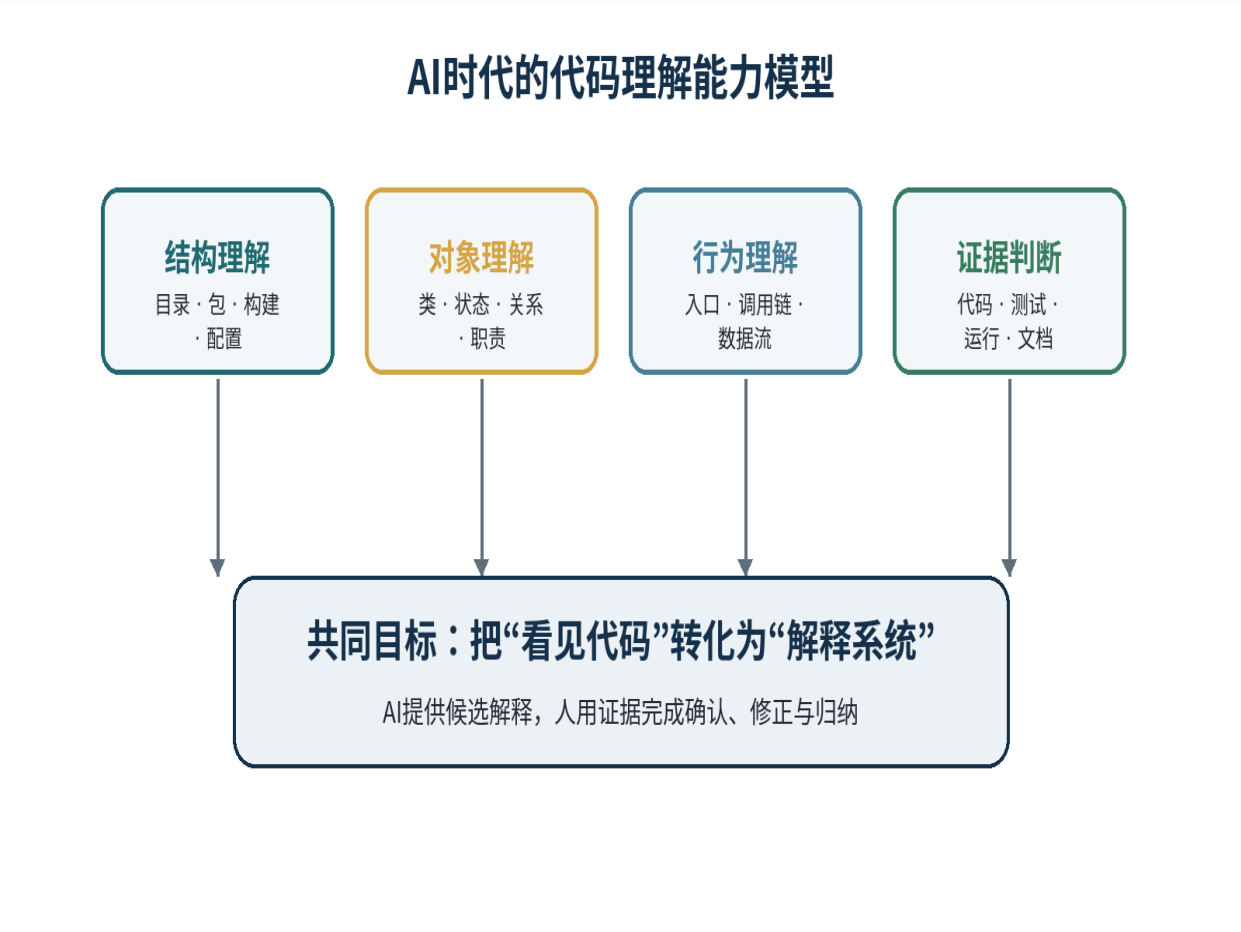
真实项目

读一个系统

- 数百个文件与多级目录
- 框架连接大量运行时行为
- 类之间关系并不写在一处
- 最重要的是先建立“地图”

代码理解的四类能力

每一次阅读都在回答四个不同问题：放在哪里？谁负责？怎样运行？凭什么相信？



学习目标

15 min 完成陌生Java仓库的第一轮结构侦察

5+ 识别至少5个核心领域对象及关系

1条 完整追踪一条真实业务调用链

3类 用源码、测试、运行结果交叉验证AI

4件 输出项目地图、类图、调用链图、证据矩阵

最终标准

不是“AI给了一个解释”，而是“我能指出代码证据，并说明这个解释为何成立”。

路线

1

仓库侦察

先看森林

2

技术栈与启动链

找到系统起点

3

领域对象

建立对象模型

4

业务调用链

让程序“动起来”

5

AI代码理解

提出假设并核验

6

测试验证

读行为边界

7

GitHub与Issue

进入维护现场

8

综合报告

把代码压缩成模型

为什么选择 Spring PetClinic?

真实，但没有大到失控

- 长期维护的Spring官方示例应用
- 业务对象清楚：Owner / Pet / Vet / Visit
- 有Controller、Repository、验证、测试等完整要素
- 支持Maven / Gradle，适合课堂复现
- 能够观察“框架自动连接”和“显式对象关系”

不是为了学完整Spring

框架知识只在帮助理解当前代码时讲解。

我们关心的是：
- 如何找到入口
- 如何识别对象
- 如何追踪调用
- 如何验证AI
- 如何进入维护场景

版本基准：教材制作时的官方主仓库（2026-09-23）

实训环境与工具分工

JDK 17+

运行Java项目

IntelliJ IDEA / VS Code

阅读、导航、断点

Git

获取仓库与查看历史

Maven / Gradle Wrapper

构建与启动

GitHub

符号导航、Issue、提交历史

AI助手

解释、归纳、生成候选结构

工具原则

AI负责“提出解释”；IDE / GitHub / 测试 / 运行负责“提供证据”。

01

仓库侦察：建立项目全景

在不深入任何业务方法之前，先知道“什么值得看、什么可以先忽略”。

第一天进入陌生项目，最常见的错误

错误方式

从 src/main/java 的第一个文件开始逐行阅读；遇到不懂的类就继续点进去；几十分钟后知道很多局部，却不知道系统整体。

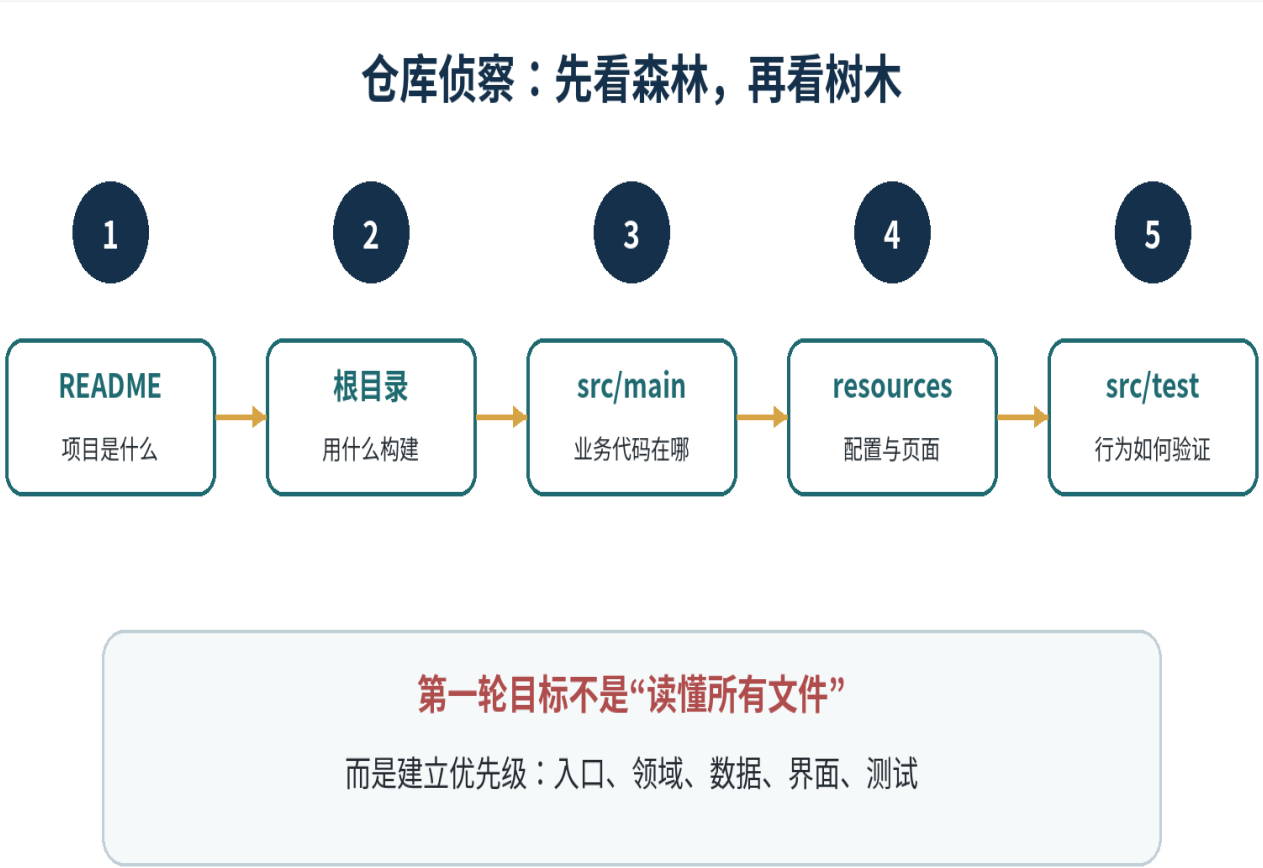
更有效的方式

先观察根目录、README、构建文件和测试位置；识别项目类型、技术栈、入口、核心包，再决定深入哪条业务链。

原则：先看森林，再看树木

Repository Reconnaissance = 用最少时间获得最大范围的结构信息

仓库侦察路线



从仓库根目录出发，逐步缩小范围，而不是直接钻进细节。

根目录快速扫描：

ROOT

```
spring-petclinic/
├─ pom.xml / build.gradle
├─ mvnw / gradlew
├─ src/
│  ├─ main/
│  └─ test/
├─ .github/
└─ README.md
```

从名字先提出假设

- pom.xml / build.gradle → 构建与依赖
- mvnw / gradlew → 项目自带构建包装器
- src/main → 生产代码与资源
- src/test → 可执行行为说明
- README → 项目入口、运行方式、约定
- .github → 协作、CI、Issue模板等

15分钟仓库侦察任务

0–3 min

只看根目录
写下5个高价值文件/目录

3–7 min

读README
确认启动方式与主类

7–11 min

看构建文件
把依赖按职责归类

11–15 min

看src/main / src/test
画第一张项目地图

提交物

一张手绘或电子“项目第一张地图”：至少标出构建、入口、业务代码、资源、测试五个区域。

AI第一次介入：只让它“画地图”

PROMPT

这是一个 Java Spring Boot 开源项目。
请暂时不要逐行解释代码。

1. 哪些目录最值得优先阅读?
2. 哪些文件决定项目如何启动?
3. 哪些目录属于业务代码、配置、资源和测试?
4. 如果只有30分钟，应按什么顺序阅读?

每个判断请说明需要到哪个文件或目录验证。

为什么这样问?

限定范围：只做结构分析
要求证据：答案必须可验证
允许不确定：避免“知道一切”的幻觉

不要这样问

“请告诉我这个项目所有代码是什么意思。”
范围过大，AI只能给表面化总结。

1检查点：你的“第一张地图”是否合格?

- 能指出构建文件
- 能指出程序入口
- 能区分业务代码与资源
- 能指出测试目录
- 至少有3条判断写明验证位置

15
分钟

5
关键区域

3+
证据

02

识别技术栈与启动链

找到“系统从哪里开始”，并防止把通用架构模板误当成当前仓库事实。

从构建文件识别技术栈：先分类，不逐个背依赖

Web

HTTP / MVC / 页面请求

Data

Repository / ORM / 数据库

Template

服务端页面模板

Validation

输入校验

Test

单元/集成/Mock

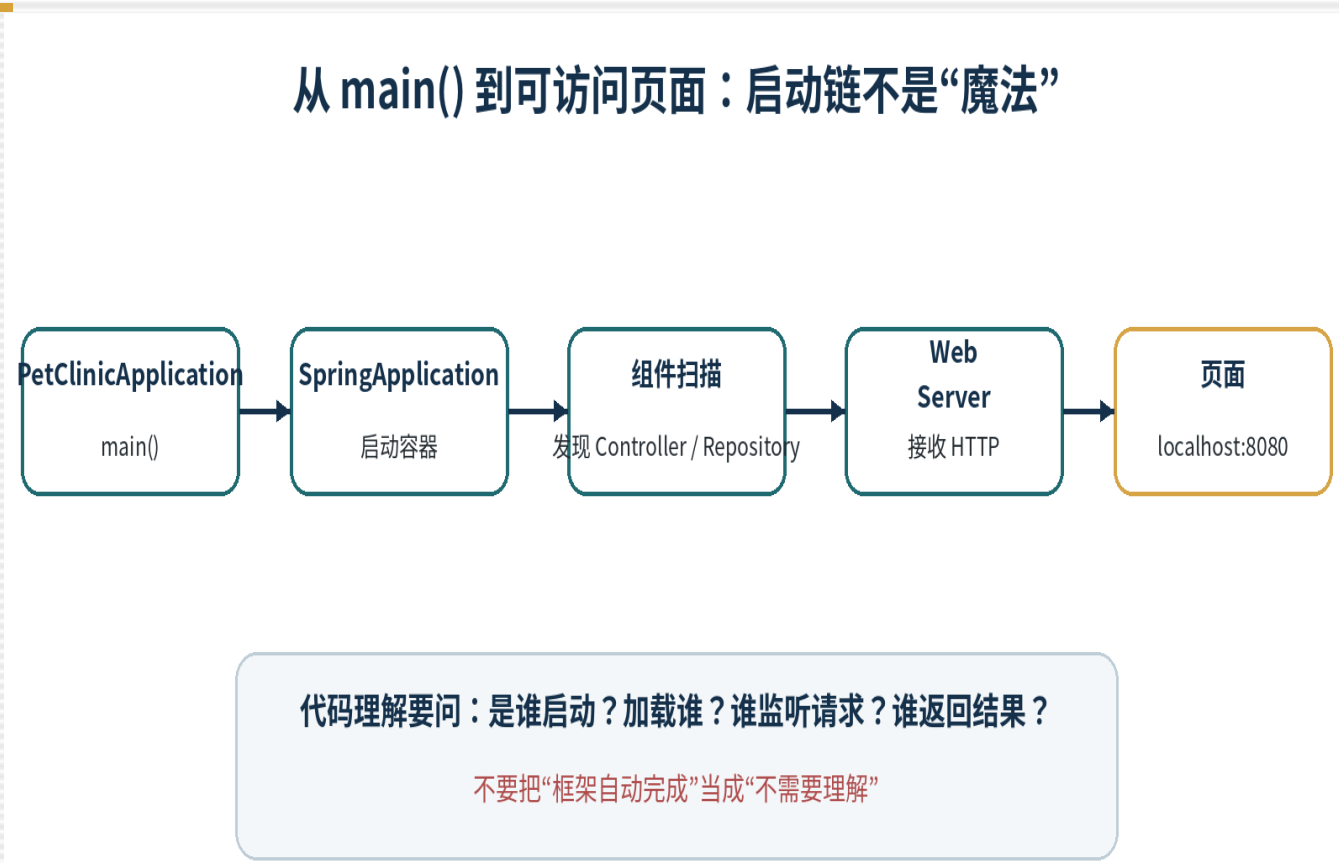
Dev

开发辅助与热更新

第一轮目标

回答“项目大致用哪些技术完成哪些职责”，而不是逐个研究每一个依赖的API。

从 main() 到页面：启动链



启动链解释“系统怎样被组织起来”，不等同于某个具体业务请求的调用链。

PetClinicApplication：只读最关键的三行

```java
@SpringBootApplication
public class PetClinicApplication {
    public static void main(String[] args) {
        SpringApplication.run(PetClinicApplication.class, args);
    }
}
```

要解释的不是语法，而是控制权

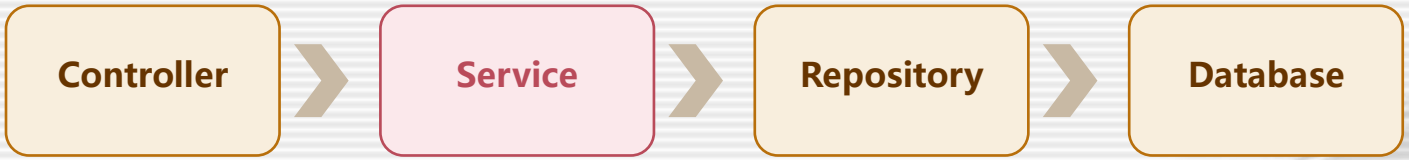
- main() 是显式入口
- SpringApplication.run() 把控制权交给框架
- 框架随后扫描、创建组件、启动Web环境

最小充分理解

你不必在此刻解释Spring容器所有内部机制；只需知道“谁接管了后续工作”。

警惕“常见架构幻觉”

AI看到 Spring Boot，可能自动补出：



当前仓库事实

至少在“新增Owner”这条路径中，OwnerController 直接依赖 OwnerRepository。不要因为“标准三层架构”常见，就自动补出独立Service层。

验证动作

搜索 service 包 / @Service / *Service 类；再查看 Controller 构造器实际依赖。

验证一个架构判断：四步搜索法

1 搜包

是否存在 service/ 目录?

2 搜类型

是否存在 *Service 类或接口?

3 搜注解

是否存在 @Service?

4 看依赖

Controller 构造器实际注入了谁?

结论写法

“未发现独立Service层”比“这个项目没有Service思想”更谨慎。前者描述代码事实，后者可能过度推断设计意图。

03

从包结构进入领域对象

把数百个文件压缩成少数核心业务对象，并准确区分对象关系。

名词法：从业务语言寻找领域对象

Owner

宠物主人

Pet

宠物

Vet

兽医

Visit

就诊记录

PetType

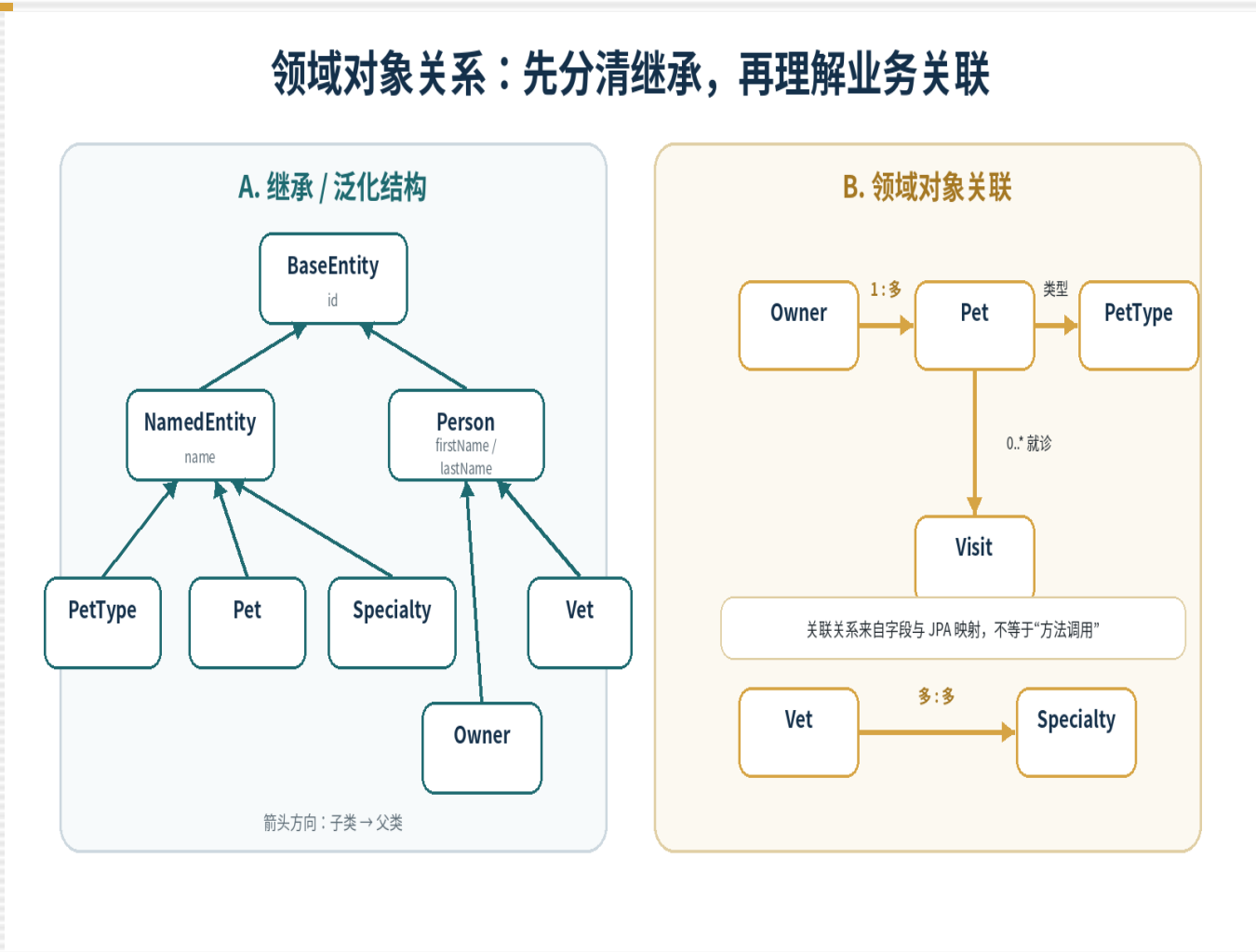
宠物类型

Specialty

专业领域

规则：先列出业务名词 → 再确认哪些名词真正被建模成类

领域对象关系：先识别共性，再识别关联



类图用于理解对象与抽象关系；不要把它直接等同于数据库ER图。

不要把所有箭头都叫“调用”



继承、关联/包含、依赖、调用描述的是不同层面的关系。

四种关系怎么判？看“证据形态”

继承	A is-a B	extends / implements	PetType → NamedEntity
关联/包含	A has-a B	字段 / 集合	Owner 持有 pets
依赖	A needs B	构造器/参数/局部对象	Controller 依赖 Repository
调用	A invokes B	方法体中的实际调用	repository.save(owner)

判断关系时，先找证据，再画箭头。

类责任卡：把一个类压缩成六个问题

1. 它代表什么对象/责任?
2. 保存哪些状态?
3. 公开哪些行为?
4. 继承自谁 / 被谁继承?
5. 持有哪些对象 / 依赖哪些协作者?
6. 修改它，哪些功能可能受影响?

AI辅助类分析：五问法

PROMPT

请从以下五个角度分析这个 Java 类：

1. 它代表现实世界中的什么对象或责任?
2. 它保存什么状态?
3. 它能够执行什么行为?
4. 它和哪些类存在继承、组合、关联或依赖?
5. 如果删除或修改这个类，哪些功能可能受到影响?

所有结论请指出对应的代码证据。

输出要求

结论 + 代码位置 + 证据
而不是“这是一段负责XXX的代码”。

加一道反证

“哪些结论只是合理推测，而不是当前代码能直接证明？”

小练习：下面三条关系分别是什么？

Owner extends Person ?

Owner 中有 List<Pet> ?

OwnerController 构造器接收 OwnerRepository ?

答案

① 继承 ② 关联/包含 ③ 依赖。真正的“调用”还要到方法体里找实际执行。

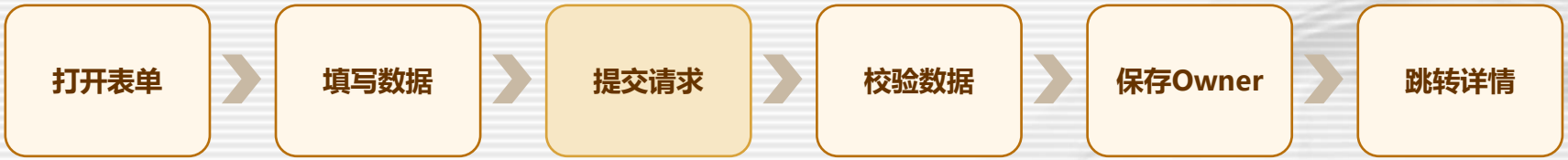
04

追踪一个真实业务调用链

从“类之间有什么关系”进入“程序在一次用户操作中怎样运动”。

大型项目不要“读完”，要“选一条线追下去”

选择业务场景：新增 Owner



为什么选用户动作?

因为它天然有“开始”和“结束”，可以把散落在多个类中的代码组织成一条可验证的执行路径。

先找 Web 入口：GET 和 POST 是两条不同路径

OWNERCONTROLLER

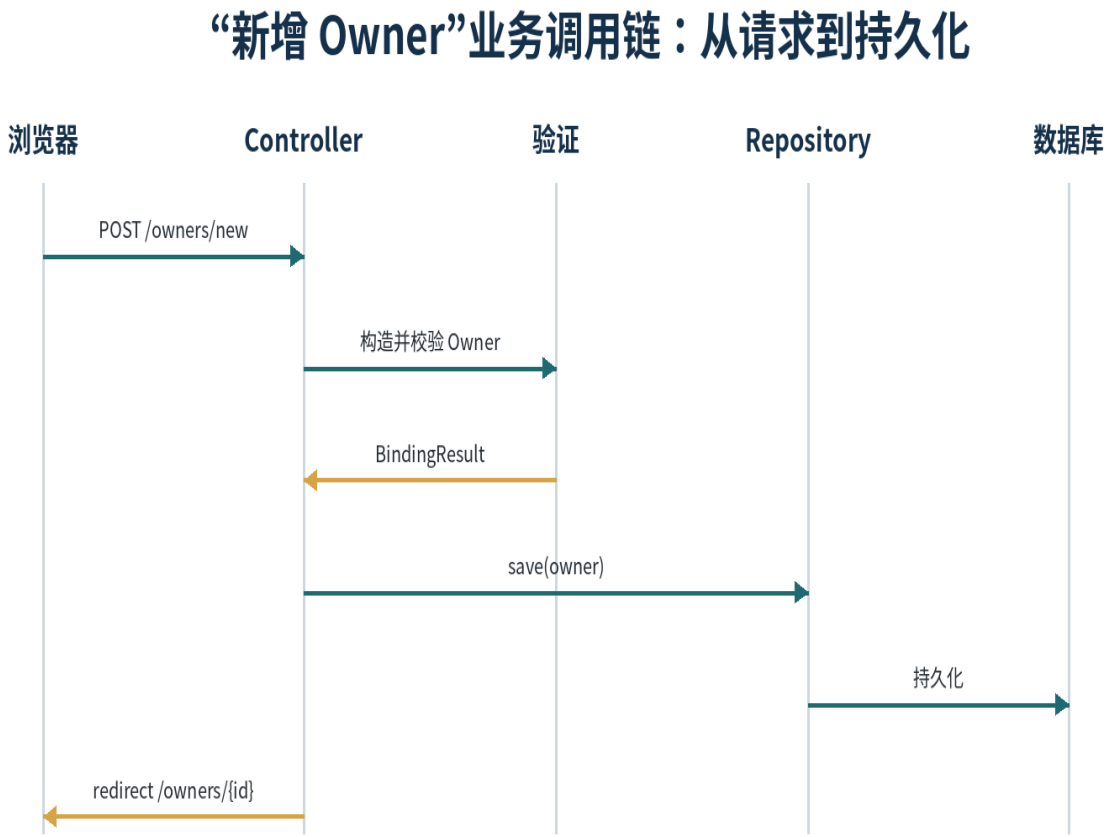
GET /owners/new
→ 显示新增表单

POST /owners/new
→ 接收表单数据
→ 校验 Owner
→ ownerRepository.save(owner)
→ redirect:/owners/{id}

阅读时问四个问题

- 哪个URL进入?
- 参数如何变成对象?
- 失败时走哪条路径?
- 成功后为什么redirect?

“新增 Owner”业务调用链



一次请求把浏览器、Controller、验证、Repository和数据库连接成动态行为。

“入口—数据—判断—调用—返回”五步法

1	2	3	4	5
入口	**数据**	**判断**	**调用**	**返回**
哪个请求/方法进入?	参数怎样成为对象?	有哪些验证/分支?	调用了哪个协作者?	返回视图、对象还是重定向?

这个五步法可以迁移到绝大多数 Web 业务方法。

静态追踪 + 断点运行：两种证据相互补充

静态阅读告诉你

“代码可能怎样走”

- 看映射
- 看方法体
- 看调用关系
- 看分支条件

运行/调试告诉你

“这一次实际怎样走”

- 在方法入口断点
- 在 save() 前后断点
- 观察对象字段变化
- 核对实际分支

关键观察：Owner 的 id 何时从空值变为生成后的值?

调用链证据表：把“感觉”变成可复核记录

1	OwnerController	POST /owners/new	映射注解
2	参数绑定	Owner对象被构造	方法参数/调试
3	Validation	错误进入BindingResult	验证注解/测试
4	Repository	save(owner)	方法调用
5	Redirect	/owners/{id}	返回值

建议：学生报告中至少保留 1 张这样的证据表。

05

AI辅助代码理解：从答案生成到证据协作

AI最有价值的能力不是替你读代码，而是加速提出结构化假设。

AI最适合做什么？不适合做什么？

适合

- 总结一个类的职责
- 比较两个实现的差异
- 解释陌生注解/框架概念
- 把局部代码压缩为调用链草图
- 从测试归纳行为边界

不应直接相信

- “这个项目一定采用某架构”
- “这个类肯定由某模块调用”
- “某设计模式一定存在”
- “某方法运行时一定走这条分支”
- 基于旧版本仓库的具体结论

AI辅助代码理解的证据闭环



AI输出不是终点；真正的闭环必须回到代码、测试和运行事实。

四类高价值提问

结构

“如果只有30分钟，哪些目录最值得优先看？”

责任

“这个类的责任、状态、协作者分别是什么？”

行为

“请按执行顺序追踪这个请求，但每一步标出文件位置。”

反证

“哪些结论是推测？怎样用当前仓库证伪？”

低质量提示词 vs 高质量提示词

低质量

“解释一下这个项目。”

问题：范围过大；没有输出结构；不要求证据；AI容易给通用答案。

高质量

“只分析新增Owner路径。按请求→参数绑定→验证→Repository→返回顺序输出；每步写类、方法、文件；不能确认时标记‘需验证’。”

优势：范围清晰、输出可检查、允许不确定。

提示词的目标：不是让AI“说得更多”，而是让输出“更容易被验证”。

AI答案的证据等级

A **运行结果 / 测试结果** 这次真的这样发生了

B **源代码 / 配置** 当前仓库直接写明

C **官方 README / 文档** 作者明确说明的使用与约定

D **AI解释** 有用，但必须验证

课堂规则

重要结论至少要回到 B 级；涉及实际行为时尽量升级到 A 级。

案例：AI说“这里应该有 Service 层”怎么办?

AI候选解释

假设

“典型 Spring MVC 项目会采用 Controller → Service → Repository。”

仓库核验

搜索 service 包、@Service 和 *Service；查看 OwnerController 构造器，发现直接依赖 OwnerRepository。

修正结论

“在当前新增Owner路径中未发现独立Service层，Controller直接调用Repository。”

AI使用日志：把“用了AI”记录成可分析过程

			结果
项目结构	建议先读README与构建文件	实际查看	成立
架构层次	存在Service层	全仓搜索+构造器	被纠正
调用链	save后重定向	源码+测试	成立
对象关系	Owner包含Pet集合	类字段	成立

实训报告至少记录 1 个“AI被代码纠正”的案例。

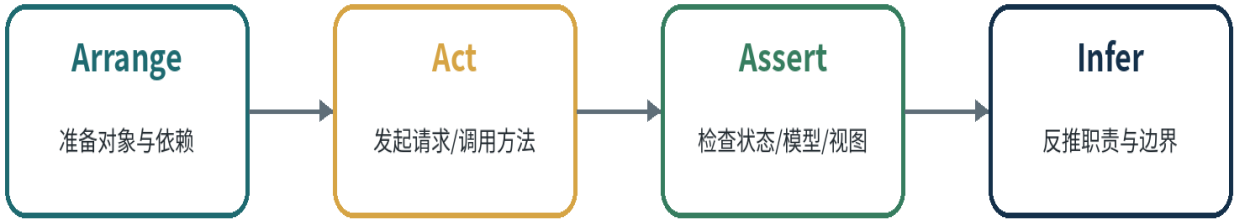
06

用测试验证代码理解

测试不是“最后再跑一下”，而是理解系统行为和边界的可执行说明书。

测试是“可执行说明书”

测试代码：把“我认为”变成“可执行证据”



阅读测试时，不只看“是否通过”

更要看：作者认为什么输入、输出和异常属于正确行为

源码告诉你“怎样实现”；测试告诉你“应该出现什么结果”。

用 Arrange—Act—Assert 反推职责

Arrange	Act	Assert	Infer
准备对象与依赖	**触发请求或方法**	**检查结果**	**反推职责与边界**
测试在构造怎样的世界?	真正执行了什么?	作者认为“正确”是什么?	哪些行为不能被修改破坏?

关键转变

不要问“这个测试用了什么API”，先问“这个测试描述了什么行为”。

只看成功路径，会错过一半系统行为

成功路径

- 合法字段
- 验证通过
- Repository保存
- 跳转到详情页

失败路径

- 缺少地址/电话等字段
- BindingResult有错误
- 不执行保存
- 返回表单并显示错误

维护中最容易被破坏的，往往正是失败路径。

设计一个新增测试：先写“测试意图”，再写代码

1. 姓名包含纯数字或特殊字符

2. 字段超长

3. 查询不存在的Owner

4. URL中的ownerId与表单对象不一致

先写一句话

“当 _____ 时，系统应该 _____，并且不应该 _____。”

再找证据

这个边界应该由验证、Controller还是Repository负责？已有测试覆盖了吗？

07

GitHub代码导航与 Issue 分析

从“阅读已有设计”进入“定位真实问题—理解影响—形成修复假设”的维护场景。

符号导航：比“全文搜字符串”更接近真实关系

Go to definition

这个类/方法在哪里定义？

Find references

谁真正引用了它？

Call hierarchy

调用关系怎样展开？

Search code

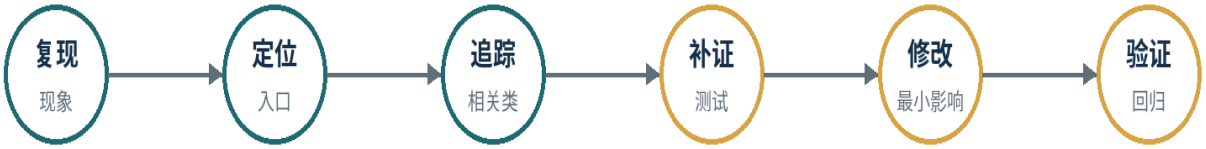
快速定位注解、包、模式

原则

能用“符号关系”回答的问题，不要只靠字符串搜索。

从 Issue 到代码的最小维护闭环

从 Issue 到代码：真实维护任务的最小闭环



示例：名称校验问题

Issue → NamedEntity / Owner / Pet → 验证规则 → 测试用例 → 回归

Issue不是“看热闹”，而是把业务现象、代码位置、测试和修复连接起来。

Issue 分析卡：六个问题定位一个真实问题

1 用户看到的现象是什么?

2 如何最小化复现?

3 涉及哪些核心类?

4 公共父类/验证规则是否相关?

5 哪些现有测试与问题有关?

6 最小修复应该改哪里，并补什么测试?

再向后看一步：代码历史告诉你“为什么变成这样”



只选一次有意义的修改

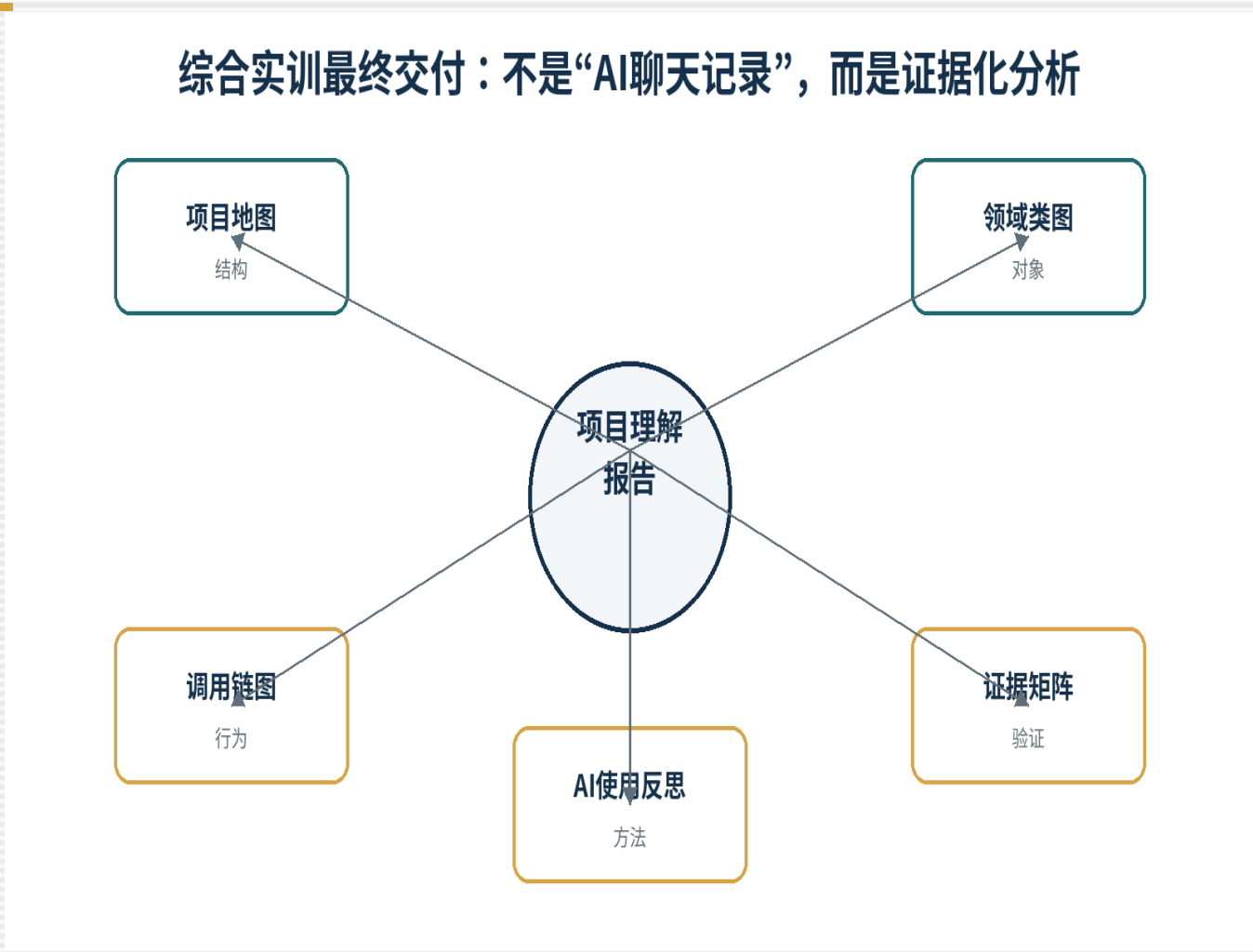
看 commit / blame：修改前后职责是否变化？测试是否同步变化？提交信息有没有解释设计动机？

08

综合实训：输出项目理解报告

把代码压缩成少数可复用模型，并为重要结论附上证据。

综合实训最终交付物



项目地图 + 领域类图 + 业务调用链 + 证据矩阵 + AI反思。

项目理解报告：建议结构

1 **项目简介** 解决什么问题 / 技术栈

2 **项目地图** 目录、入口、配置、测试

3 **领域对象** 核心类、继承、关联

4 **业务调用链** 至少一条端到端路径

5 **测试与边界** 成功/失败路径

6 **AI证据日志** 至少1次被代码纠正

7 **维护观察** Issue或历史修改

8 **学习反思** 方法如何迁移到新项目

评分量表：高分来自“证据化理解”，不是AI生成得多

项目结构分析	15	地图准确，能解释阅读顺序
核心类与关系	15	关系类型准确，有代码证据
调用链分析	20	端到端、顺序清楚、包含失败路径
代码证据与验证	15	重要判断可追溯
测试理解	10	能从测试反推边界
AI使用方法	10	有验证、有纠错、有反思
图表与报告	15	结构清楚、可复用、表达专业

口头答辩：5个问题检验你是否真的读懂

1 如果删掉一个核心类，最先坏掉的功能是什么？依据是什么？

2 你画的类图里，哪条关系最容易被误判？

3 调用链中，哪一步是框架完成，哪一步是项目代码完成？

4 AI在哪个问题上给过你“看似合理但实际不准确”的答案？

5 如果项目升级后目录变化，你的方法中哪些部分仍然有效？

可直接复用的 AI 提示词卡

结构地图

只分析项目结构；给出优先阅读顺序；每个判断标出验证文件。

类责任

从对象/状态/行为/关系/影响五个角度分析；附代码证据。

调用链

按执行顺序输出类、方法、文件；不能确认处标“需验证”。

测试理解

按Arrange—Act—Assert总结，并反推被测代码职责和边界。

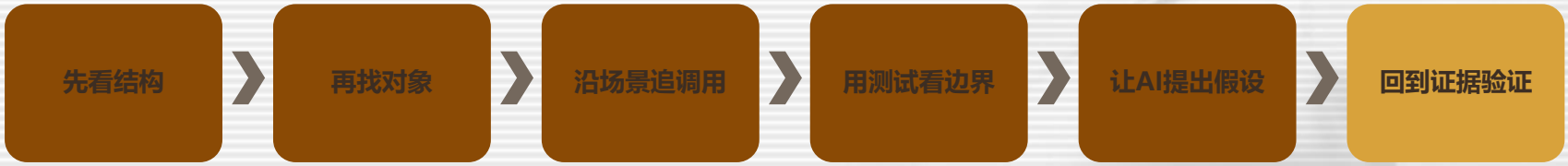
反证检查

指出哪些结论只是推测，并给出最短验证路径。

变更影响

如果修改这个类/方法，哪些调用者、测试和业务场景最可能受影响？

学习的不是 PetClinic，
而是一套进入陌生项目的方法



AI降低“进入陌生代码”的成本，但不会替代判断。
可靠的程序理解，始终要重新连接到代码、测试和运行事实。

面向对象知识，在AI时代从“写类的规则”变成“理解复杂软件的语言”。

参考资料与版本说明

- spring-projects/spring-petclinic：Spring PetClinic 官方主仓库（教材版本基准：2026-09-23）
- Spring PetClinic README：本地运行、数据库配置、IDE使用与主类说明
- pom.xml / build.gradle：构建元数据、Java版本与依赖
- OwnerController.java：Owner相关请求处理与Repository依赖
- OwnerControllerTests.java：Controller行为测试
- GitHub Docs：Navigating code on GitHub（定义、引用、符号导航）
- Spring PetClinic Issue #2554：Owner/Pet名称校验问题（维护案例）

课堂使用提醒

开源项目持续更新。若类名、目录、Issue状态发生变化，应以当前官方仓库为准；这恰好也是训练“版本意识”的机会。